

Encryption Method

1. Install Terraform on your system and use it to write a basic configuration file main.tf that provisions a single AWS EC2 instance named 'music-streaming-server'.

ANS

- I installed Terraform on my system and verified the installation using:
 - terraform --version
- I then created a file named main.tf to provision an AWS EC2 instance named music-streaming-server.

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "music_streaming_server" {  
    ami      = "ami-xxxxxxxxxxxxxxxxxxxxx"  
    instance_type = "t2.micro"  
    tags = {  
        Name = "music-streaming-server"  
    }  
}
```

- Replace ami-xxxxxxxxxxxxxxxxxxxxx with a valid Amazon Linux AMI ID for the us-east-1 region.

2. Refactor your main.tf by splitting the EC2 resource into a separate file called ec2.tf and use a variables.tf file to define the instance type and AMI as input variables.

Hint: This helps make your configuration modular and reusable.

ANS

- I separated the EC2 configuration into ec2.tf and created a variables.tf file to define the AMI ID and instance type as input variables. This makes the Terraform configuration easier to maintain and reuse.
 - **variables.tf**
 - variable "instance_type" {
 - description = "EC2 instance type"
 - type = string
 - default = "t2.micro"

1) ec2.tf

```
resource "aws_instance" "music_streaming_server" {  
    ami          = var.ami_id  
    instance_type = var.instance_type  
    tags = {  
        Name = "music-streaming-server"  
    }  
}
```

2) main.tf

```
provider "aws" {  
    region = "us-east-1"  
}
```

terraform.tfvars

```
ami_id      = "ami-xxxxxxxxxxxxxxxxxxxx"  
instance_type = "t2.micro"
```

The terraform.tfvars file provides the actual values for the variables.

3. Add a Terraform resource to your configuration that provisions an AWS RDS MySQL database instance named 'playlist-db' in a separate db.tf file, and use outputs.tf to display the database endpoint after apply.

ANS

- I created a separate file named db.tf to provision an AWS RDS MySQL database named playlist-db. I also created outputs.tf to display the database endpoint after Terraform applies the configuration.

1) db.tf

```
resource "aws_db_instance" "playlist_db" {  
    identifier      = "playlist-db"  
    engine          = "mysql"  
    engine_version  = "8.0"  
    instance_class  = "db.t3.micro"  
    allocated_storage = 20
```

```

db_name = "playlistdb"
username = "admin"
password = "ChangeMe123!"
skip_final_snapshot = true
tags = {
    Name = "playlist-db"
}
}

```

2) outputs.tf

```

output "database_endpoint" {
    description = "RDS database endpoint"
    value      = aws_db_instance.playlist_db.endpoint
}

```

- After applying the Terraform configuration:
- terraform apply
- Terraform displays the database endpoint.

4. Initialize a new Git repository in your Terraform project folder, commit all your configuration files, and write a README.md that explains how to deploy your infrastructure using Terraform.

Hint: Include setup steps and any variable values that need to be provided.

ANS

- I initialized a Git repository in my Terraform project directory and committed the Terraform configuration files.
 - **Initialize Git**
 - git init
 - **Add Files**
- git add main.tf ec2.tf variables.tf terraform.tfvars db.tf outputs.tf
 - **Commit Files**
- git commit -m "Add Terraform AWS infrastructure"
- I also created a README.md file explaining how to deploy the infrastructure.

5. List three best practices for managing Terraform configurations in a real-world team project, and briefly explain how each practice helps avoid problems when multiple developers are working together.

ANS

1) **Use Git for Version Control:**

Store Terraform files in a Git repository so multiple developers can track changes, work on separate branches, and review changes before merging.

2) **Use Variables and Modules:**

Use variables and reusable modules instead of hard-coding values. This makes the configuration easier to maintain and reuse across different environments.

3) **Use Remote State Management:**

Store the Terraform state in a shared remote backend so all team members work with the same state and avoid conflicts or inconsistent infrastructure.